Prueba Técnica Dev. Webflux

Fecha: 2026-10-29 **Nivel:** Senior / Arquitecto **Tecnología:** Java — Spring Boot 3, Spring WebFlux, Project Reactor (Mono/Flux), Kafka, Resilience4j, Spring AI o LangChain4j, R2DBC, Docker, Kubernetes **Plazo de entrega:** 3 días

---

Contexto del Problema

VidaCruz es un grupo de salud que opera 12 salas de urgencias en el país. El triaje se realiza hoy de forma manual: la enfermera evalúa los signos vitales del paciente en la recepción y escribe el nivel de prioridad en una hoja. Esto genera tres problemas crónicos: la clasificación varía según la enfermera (hay reclamos por pacientes graves que esperan horas), no hay trazabilidad de qué pasó con cada paciente y su tiempo de espera, y los médicos no tienen a la mano los protocolos clínicos internos cuando deben decidir rápido.

La dirección ha encargado un **sistema de triaje digital con asistente clínico** compuesto por al menos dos microservicios en Java reactivo:

1. **Servicio de Triage**: registra pacientes con sus signos vitales, los clasifica automáticamente según reglas clínicas objetivas, vigila los tiempos de atención (SLA) por nivel y escala automáticamente a los pacientes que los superan, con auditoría clínica completa.
2. **Servicio de Asistente Clínico**: permite a los médicos consultar los protocolos clínicos internos en lenguaje natural, usando IA generativa. Dada la naturaleza clínica, el asistente **nunca puede inventar contenido**: toda respuesta debe estar sustentada en los protocolos del corpus oficial, con cita de la fuente.

El sistema debe demostrar un manejo serio de la programación reactiva (sin llamadas bloqueantes en el event loop), integración asíncrona con Kafka, resiliencia ante la falla del proveedor de IA y observabilidad.

Importante: Este es un sistema de backend (APIs + eventos). No se requiere frontend.

Requerimientos Funcionales

RF-01: Registro y clasificación automática de pacientes

`POST /triage/pacientes` — cuerpo con: nombre completo, edad, frecuencia cardíaca (bpm), presión sistólica (mmHg), presión diastólica (mmHg), SpO2 (%), temperatura (°C), dolor (0-10), GCS (3-15),

queja principal (texto).

- Validar los rangos de datos (ver **Validaciones de datos**). Valor fuera de rango → respuesta `400` con mensaje claro indicando el campo inválido.
- Clasificar automáticamente al paciente según la **RN-01**.
- La respuesta debe incluir: `id`, `nivel` clasificado, `slaMinutos` según **RN-02**, `estado: "REGISTRADO"` y `fechaRegistro` (ISO 8601).
- Publicar el evento `paciente_registrado` en el topic Kafka `triaje.eventos`.

RF-02: Flujo de estados clínicos

`PATCH /triage/pacientes/{id}/estado` — cuerpo `{ "estado": "..." }`

- Estados: `REGISTRADO`, `EN_CONSULTA`, `ALTA`.
- Transiciones válidas: `REGISTRADO → EN_CONSULTA`, `REGISTRADO → ALTA` (alta directa), `EN_CONSULTA → ALTA`.
- Transición inválida (ej. `ALTA → EN_CONSULTA`) → respuesta `409` con mensaje descriptivo.
- Cada transición válida debe: registrar entrada de auditoría (**RN-04**) y publicar el evento correspondiente en `triaje.eventos` (`paciente_en_consulta` o `paciente_alta`).
- `GET /triage/pacientes/{id}` → datos completos del paciente, estado actual, flags (ej. escalado) y tiempo en el estado actual.

RF-03: Monitoreo de SLA y escalamiento automático

- Un proceso de fondo (scheduler, consumidor o equivalente — tu decisión) debe evaluar periódicamente a los pacientes activos (estado `REGISTRADO`).
- Si un paciente superó su SLA (**RN-02**) sin haber entrado a consulta: marcarlo como `ESCALADO` (flag persistente), publicar el evento `paciente_escalado` en `triaje.eventos` y registrar auditoría.
- Cumplir la **RN-03** (idempotencia del escalamiento).
- El intervalo de evaluación y los SLA deben ser **configurables** (propiedades de aplicación) para permitir probar el escalamiento sin esperar decenas de minutos.

RF-04: Asistente clínico con IA (servicio de Asistente)

Parte A — Consulta con RAG: `POST /asistente/consulta` — cuerpo `{ "pregunta": "..." }`

- Respuesta: `{ "respuesta": "...", "fuentes": [ { "protocolo": "PROTO-XXX", "seccion": "..." } ] }`.
- Usar RAG (Retrieval-Augmented Generation) sobre el **corpus de protocolos** proporcionado (ver Datos de Referencia). Toda respuesta debe cumplir la **RN-05** (cita obligatoria, sin alucinaciones).

- Si no hay protocolo relevante en el corpus, el asistente debe responder de forma explícita que **no existe un protocolo disponible** para la pregunta y no debe generar contenido clínico.

Parte B — Estado de la urgencia en tiempo real: `GET /asistente/estado-urgencias`

- Devuelve la cantidad de pacientes activos (todos los que no estén en `ALTA`) por nivel de triaje.
- El asistente **debe reconstruir este estado consumiendo el topic** `triaje.eventos` (no haciendo llamadas a la API de triaje), persistiendo la vista en Redis o en una base NoSQL a tu elección.

RF-05: Reporte de operación

`GET /triage/reportes/operacion?desde=...&hasta=...` (rango de fechas ISO 8601)

- Agrupado por hora, debe devolver: total de pacientes registrados, conteo por nivel de triaje, tiempo promedio de espera (registro → primera entrada a consulta, en minutos) y percentil 95 (p95) de ese tiempo.
- La implementación **debe usar SQL avanzado** (window functions con `OVER`, `RANK` / `ROW_NUMBER`, agregados con filtros, subconsultas correlacionadas o equivalente) — no está permitido iterar en memoria sobre todos los registros para calcular agregados.

RF-06: Alertas de escalamiento al equipo médico ⚠️

El sistema debe alertar al equipo médico cuando un paciente es escalado.

El canal, el formato y la tecnología no están especificados. Debes decidir, implementar y documentar tu elección con su justificación (en el README o en un ADR).

> Este requerimiento está deliberadamente abierto. Tu decisión, tus supuestos y cómo los documentes forman parte de la evaluación (autonomía, ownership y tolerancia a la ambigüedad).

Reglas de Negocio

RN-01 — Clasificación por nivel de triaje. Evaluar las condiciones en el orden indicado; el **primer nivel que se cumpla** determina la clasificación. Si varias condiciones de distintos niveles se cumplen, rige el nivel de mayor gravedad.

Nivel	Nombre	Condiciones (cualquiera)
1	`CRITICO`	FC < 40 o FC > 120, o SpO2 < 90, o P. sistólica < 90, o GCS ≤ 8
2	`URGENTE`	90 ≤ SpO2 ≤ 93, o 90 ≤ P. sistólica ≤ 100, o Temperatura ≥ 38.5, o 9 ≤ GCS ≤ 13
3	`URGENTE_2`	90 ≤ FC ≤ 120, o 38.0 ≤ Temperatura < 38.5, o Dolor ≥ 7
4	`NO_URGENTE`	No cumple ninguna condición de los niveles 1-3
5	`NO_URGENTE_2`	Clasifica como nivel 4 **y** la queja principal pertenece a la lista de quejas administrativas (ver Datos de Referencia)

RN-02 — SLA por nivel (tiempo máximo desde el registro hasta entrar a consulta, y momento del escalamiento):

Nivel	SLA (minutos)
`CRITICO`	5
`URGENTE`	10
`URGENTE_2`	30
`NO_URGENTE`	60
`NO_URGENTE_2`	120

RN-03 — Idempotencia del escalamiento. Un paciente puede ser escalado **una única vez** (el segundo intento no publica otro evento ni duplica auditoría). Un paciente en estado `EN_CONSULTA` o `ALTA` nunca debe ser escalado.

RN-04 — Auditoría clínica. Todo cambio de estado o flag (registro, entrada a consulta, escalamiento, alta) debe quedar registrado con: timestamp, id de paciente, estado anterior, estado nuevo y tipo de evento. La auditoría **no debe ser borrable** a través de la API (no existe endpoint de eliminación).

RN-05 — Anti-alucinación del asistente. Toda respuesta del asistente sobre contenido clínico debe incluir al menos una fuente válida del corpus. Si la fuente no existe en el corpus, la respuesta es inválida. Si no hay coincidencia relevante, la respuesta debe ser exclusivamente el aviso de "no existe protocolo disponible", sin contenido clínico generado.

Validaciones de datos (RF-01)

Campo	Rango válido
Frecuencia cardíaca	20 – 250 bpm
Presión sistólica	60 – 260 mmHg
Presión diastólica	30 – 160 mmHg, y siempre menor que la sistólica
SpO2	70 – 100 %
Temperatura	34.0 – 42.0 °C
Dolor	0 – 10
GCS	3 – 15
Edad	0 – 120 años
Nombre	3 – 100 caracteres

Datos de Referencia

Pacientes de ejemplo para pruebas (usa estos para validar tu implementación — la columna "Nivel esperado" es el resultado correcto según RN-01):

#	FC	P. Sist	P. Dias	SpO2	Temp	Dolor	GCS	Queja principal	Nivel esperado
E1	130	180	110	97	36.8	8	15	Dolor torácico	`CRITICO`
E2	95	150	95	91	37.0	4	15	Dificultad para respirar	`URGENTE`
E3	88	130	85	98	38.2	6	15	Fiebre	`URGENTE_2`
E4	72	120	80	99	36.5	2	15	Herida por cortar	`NO_URGENTE`
E5	70	115	75	99	36.4	0	15	Renovación de recetario	`NO_URGENTE_2`
E6	35	80	50	85	35.9	0	6	Inconsciente	`CRITICO`

Lista de quejas administrativas (para nivel 5): `documentos`, `recolección de resultados`, `renovación de recetario`, `vacunación`, `seguimiento`.

Corpus de protocolos clínicos (mínimo obligatorio; puedes extenderlo con protocolos propios):

- **PROTO-001 — Crisis Hipertensiva, manejo inicial**
  - *Definición:* Presión arterial ≥ 180/120 mmHg. Si hay daño de órgano diana es emergencia; si no, urgencia.
  - *Medidas iniciales:* Reconfirmar la presión tras 5 minutos de reposo. No reducir la presión más de un 25% en la primera hora. Vigilar signos de encefalopatía.
  - *Contraindicaciones:* Nifedipino sublingual. Reducción intravenosa rápida sin monitorización en cuidados intensivos.
- **PROTO-002 — Sepsis, identificación temprana (qSOFA)**
  - *Criterios:* 2 o más de: frecuencia respiratoria ≥ 22, alteración del estado mental, presión sistólica ≤ 100 mmHg.
  - *Acciones iniciales:* Lactato y hemocultivos antes del antibiótico. Antibiótico de amplio espectro en la primera hora. Fluidoterapia con 30 ml/kg de cristaloides si hay hipotensión o lactato ≥ 4.
- **PROTO-003 — Deshidratación pediátrica**
  - *Clasificación:* Leve (pérdida 3-5% del peso), moderada (5-8%), severa (>8% o alteración del estado mental).
  - *Rehidratación:* Leve o moderada sin vómito → solución oral 50 ml/kg en 4 horas. Severa → vía intravenosa, bolo de 20 ml/kg.

Consultas de ejemplo para el asistente (usarlas en tus tests):

1. *"¿Cuáles son las medidas iniciales ante una crisis hipertensiva?"* → Debe responder con contenido del PROTO-001 y citarlo en `fuentes`.
2. *"¿Cuándo se administra el primer antibiótico en sepsis?"* → Debe responder con contenido del PROTO-002 y citarlo.
3. *"¿Cuál es el tratamiento de primera línea para la diabetes tipo 2?"* → **No hay protocolo en el corpus**: debe responder que no existe protocolo disponible, sin generar contenido clínico.

Requerimientos Técnicos

- La **arquitectura es de libre elección** del candidato: cómo organizas los módulos, las capas internas, los patrones (hexagonal, limpia, por capas, etc.) y el formato de los mensajes es tu decisión. **Se evaluará la decisión arquitectónica como indicador de capacidades de diseño.**

- El sistema debe constar de **mínimo 2 microservicios** en Java reactivo: Triage y Asistente Clínico. Una descomposición adicional (si la consideras) es tu decisión y debe estar justificada.
- **Stack reactivo obligatorio:** Spring Boot 3 + Spring WebFlux con `Mono` / `Flux`, y driver reactivo (R2DBC) para PostgreSQL. Las llamadas bloqueantes en el event loop serán penalizadas.
- **Kafka** para la integración entre servicios: topic `triaje.eventos` con los eventos `paciente_registrado`, `paciente_escalado`, `paciente_en_consulta`, `paciente_alta`. El formato de mensaje (JSON, Avro, etc.) es libre — documentarlo.
- **Resilience4j:** circuit breaker sobre las llamadas al proveedor de LLM, con fallback explícito (modo degradado: respuesta `503` con mensaje claro cuando el LLM no está disponible). El servicio de triaje **nunca** debe depender de la disponibilidad del asistente.
- **RAG:** Spring AI o LangChain4j (a tu elección). El vector store es de libre elección (in-memory, pgvector, Redis u otro) — documentarlo en un ADR.
- **Observabilidad:** probes de health/readiness, logs estructurados con correlación, métricas con Micrometer (mínimo: conteos de clasificación por nivel, escalamientos, latencia del LLM). El trazado distribuido (OpenTelemetry) es **bonus**.
- **Seguridad:** ninguna credencial hardcodeada (la API key del LLM va por variable de entorno), autenticación básica en los endpoints (JWT o API key, a tu elección — documentarla) y validación de entradas.
- **Tests:** unitarios + de integración. El LLM debe poder mockearse en tests. Se recomienda (no es obligatorio) Testcontainers o equivalente para PostgreSQL y Kafka. Cubrir como mínimo: clasificación de E1-E6, transiciones de estado válidas e inválidas, idempotencia del escalamiento, citas del RAG y el caso de "sin coincidencia sin alucinación".
- **Ejecución local:** un `docker-compose` que levete todo con un solo comando (PostgreSQL, Kafka o equivalente, Redis y ambos servicios).
- **Kubernetes:** manifests (Deployment, Service, probes) con Azure como objetivo (AKS o Azure Container Apps, a tu elección).
- **ADR:** mínimo 2 decisiones documentadas (ej: descomposición de servicios, modo degradado, vector store, formato de mensajes).

Entregables Obligatorios

1. **Repositorio GitHub** (público, o privado con acceso habilitado para el evaluador) con el código fuente completo.
2. **README.md** con: instrucciones claras para ejecutar el proyecto localmente (un solo comando), descripción de la arquitectura elegida con diagrama, tecnologías utilizadas y decisiones clave con su justificación.
3. **Tests automatizados** que validen los flujos de negocio.
4. **URL de la aplicación desplegada en Azure** con las APIs accesibles, más los manifests de Kubernetes / IaC utilizados para el despliegue.

5. **ADR** (mínimo 2).

Criterios de Evaluación

Se evaluará:

- Cumplimiento de los requerimientos funcionales y reglas de negocio (incluida la calidad del SQL avanzado del RF-05)
- Arquitectura de microservicios y diseño de la solución
- Integración reactiva (Reactor, Kafka, backpressure)
- IA: calidad del RAG, citas, anti-alucinación y modo degradado
- Resiliencia y observabilidad
- Manejo de errores y excepciones
- Seguridad de la aplicación
- Cobertura y calidad de las pruebas
- Despliegue e infraestructura (Docker, Kubernetes, Azure)
- Documentación, ADR y habilidades demostradas (ownership, autonomía, bias por la ejecución)

Notas Adicionales

- **No se responderán preguntas durante el desarrollo de la prueba.** Cualquier ambigüedad que encuentres (incluida la RF-06) es parte de la evaluación: documenta tus decisiones y supuestos.
- **Commitea progresivamente:** un historial de commits iterativo con versiones funcionando es valorado positivamente (bias por la ejecución).
- Puedes usar el proveedor de LLM y la API key que prefieras (siempre por variable de entorno, nunca commiteada).
- Puedes usar herramientas de IA para el desarrollo. El estándar esperado es el de un desarrollador senior que las utiliza: sistema funcionando, decisiones justificadas y estructura de calidad.
- El corpus mínimo de protocolos es el proporcionado; puedes extenderlo con protocolos propios.